Memoria de Prácticas

Introducción al Aprendizaje Automático

Jesús Fernández López
Rafael Carlos Schoettel Vilchez

Abril, 2026

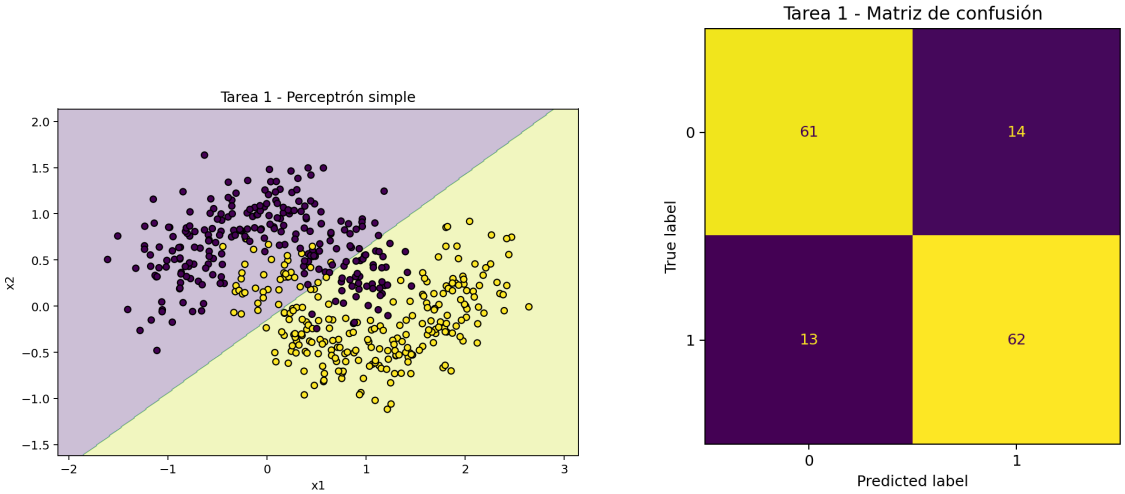
Arquitectura de Redes Neuronales: El Perceptrón Multicapa

En esta práctica se explora el funcionamiento del Perceptrón Multicapa (MLP) y su capacidad para modelar fronteras de decisión no lineales. A diferencia del perceptrón simple, el MLP aborda problemas de clasificación más complejos mediante la introducción de capas ocultas. A lo largo del documento se analizan distintos hiperparámetros, como el número de neuronas y la función de activación, y su impacto en la convergencia y generalización del modelo.

1.1. Tarea 1: La barrera de la linealidad — El Perceptrón Simple

Para iniciar el estudio, se ha evaluado el rendimiento de un Perceptrón simple (un modelo sin capas ocultas, definido con `hidden_layer_sizes=()`) aplicado al dataset sintético `make_moons`. Aunque tras el entrenamiento se alcanza una precisión en test de aproximadamente un 82 %, el análisis visual de las gráficas revela que el modelo falla de manera sistemática en ciertas regiones complejas.

1.1.1. Resultados y Análisis Estructural



(a) Frontera de decisión lineal.

(b) Matriz de confusión.

Figura 1.1: Evaluación del rendimiento en un perceptrón de una sola capa.

Al observar la frontera de decisión (Figura 1.1a), la causa del error se hace evidente. El dataset presenta dos clases distribuidas en forma de lunas entrelazadas, pero el modelo ha trazado un límite de separación completamente recto. Matemáticamente, es imposible separar estas dos clases con un solo hiperplano, lo que explica la distribución de fallos en la matriz de confusión (Figura 1.1b).

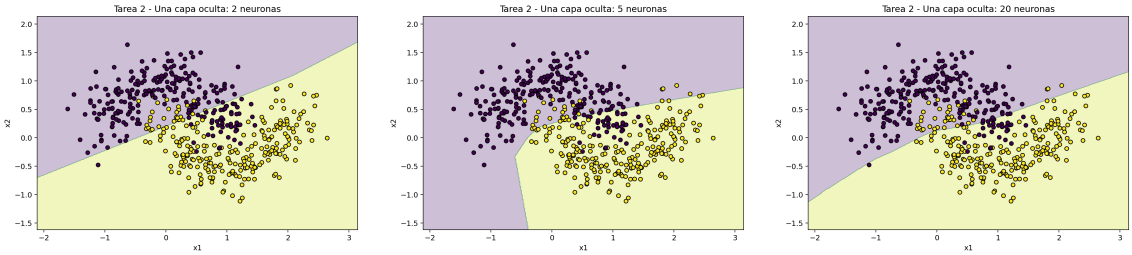
Este comportamiento no indica una falta de entrenamiento, sino una limitación inherente a la arquitectura. Al no disponer de capas ocultas, un perceptrón simple solo puede generar combinaciones lineales de las entradas, restringiendo su capacidad a la creación de fronteras lineales.

Se concluye que el modelo padece un grado severo de *underfitting* debido a su simplicidad. Para clasificar correctamente distribuciones que no son linealmente separables, es necesario dotar a la red de representaciones no lineales mediante el uso de capas ocultas.

1.2. Tarea 2: Impacto de la capacidad en la capa oculta

Una vez constatada la limitación del modelo lineal, se procede a introducir una capa oculta en el MLP para proporcionarle capacidad de decisión no lineal. Se han

evaluado tres configuraciones distintas (2, 5 y 20 neuronas) para ilustrar visualmente la adaptación de la frontera de decisión a medida que aumenta la complejidad de la red.



(a) 2 neuronas: Underfitting. (b) 5 neuronas: Transición. (c) 20 neuronas: Ajuste óptimo.

Figura 1.2: Evolución de la frontera de decisión según la densidad de la red.

Tabla 1.1: Resultados experimentales según la arquitectura.

Arquitectura	**Acc. Test**	**Iteraciones**	**Observación**
MLP(2)	0.8200	648	Capacidad insuficiente
MLP(5)	0.8400	726	Mayor esfuerzo de optimización
MLP(20)	0.8533	406	Convergencia eficiente

Configuración con 2 neuronas

Con dos neuronas, la red adquiere una ligera flexibilidad respecto al límite estático de la tarea anterior. No obstante, al observar la gráfica se aprecia que la frontera forma un ángulo sencillo en forma de pico. La capa carece de los parámetros suficientes para contornear la distribución, manteniendo la red un problema general de *underfitting*.

Configuración con 5 neuronas

Al escalar a 5 neuronas, el MLP consigue envolver ambas clases con mucha mayor efectividad. Resulta evidente en la representación que la frontera no es suavemente curva: al emplear la función de activación ReLU, la forma se delimita ensamblando segmentos de recta por cada neurona oculta. Este comportamiento resuelve las carencias iniciales, dando lugar a un contorno poligonal característico de redes con pocos nodos y activadas por ReLU.

Configuración con 20 neuronas

Finalmente con 20 neuronas, la convergencia visual es notable y se logran definir tramos liminares que asemejan una transición mucho más precisa y suave. A pesar de incrementar la capacidad de la red para una simulación bidimensional relativamente breve, el modelo derivado no incurre en *overfitting* (sobreajuste), dado que evita perfilar curvas anómalas o abarcar puntos errantes generados por el ruido paramétrico; por otra parte las métricas confirman estos supuestos marcando precisión en train (86.00 %) y test (85.33 %) prácticamente similares, apuntando a buena capacidad de generalización aunque con mejoras reducidas con respecto al ejemplo de los 5 nodos.

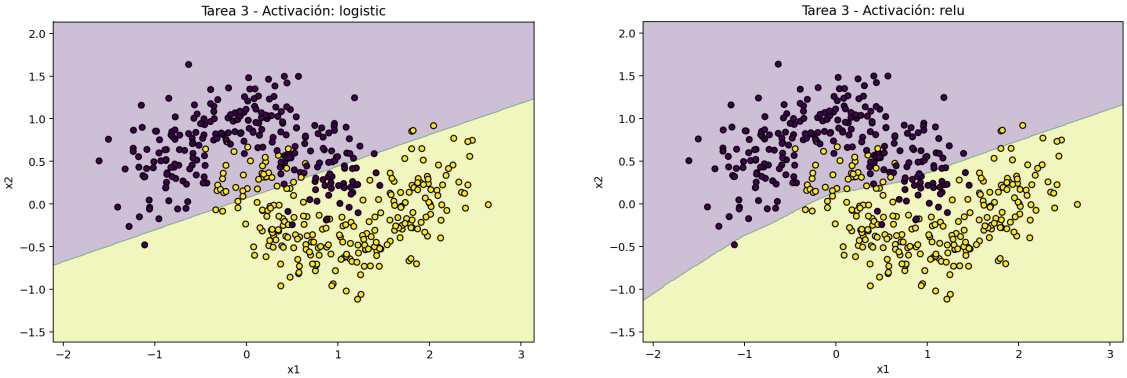
Conclusión

Se confirma visualmente cómo el agregado de nodos de red en la capa oculta soluciona la barrera lineal inicial en la práctica, adaptando el proceso natural de clasificación a formas geométricas más complejas y fidedignas (en este caso logradas superponiendo perfiles rectos aportados por las activaciones ReLU). Estructuralmente hablando se pone de manifiesto una regla clásica sobre configuraciones base: deben poseer densidad suficiente para eludir problemas de subajuste y simultáneamente ajustarse en mesura evitando sobredimensionar su consumo a expensas de beneficios ínfimos en eficacia.

1.3. Tarea 3: Dinámica de Aprendizaje — ReLU vs. Sigmoide

Manteniendo una arquitectura uniforme basada en una capa oculta de 20 neuronas, se expone ahora la comparativa entre la función logística (*Sigmoide*) y la unidad lineal rectificada (*ReLU*). El fin de este ejercicio es medir la eficacia en la separación y los tiempos de entrenamiento modificando únicamente el selector de activación de la red.

1.3.1. Fronteras de decisión



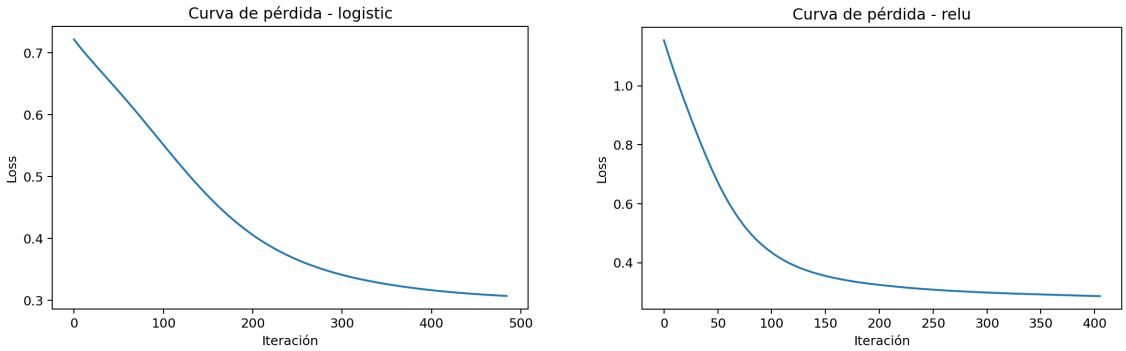
(a) Frontera con Sigmoide.

(b) Frontera con ReLU.

Figura 1.3: Comparación visual de la frontera de decisión según la activación.

Ambas funciones resuelven de forma adecuada la separación no lineal, aunque exhiben trazados característicos acordes a sus formulaciones algebraicas. La curva Sigmoide plantea una barrera orgánica como fruto de su definición dentro del intervalo $(0, 1)$, mientras que el corte de la función ReLU vuelve a imponer polígonos funcionales de transiciones estrictas y acotadas.

1.3.2. Curvas de pérdida y velocidad de convergencia



(a) Curva de pérdida con Sigmoide.

(b) Curva de pérdida con ReLU.

Figura 1.4: Comparativa de la velocidad de aprendizaje.

Tabla 1.2: Métricas comparativas por función de activación (arquitectura `(20,)`).

Activación	Acc. Train	Acc. Test	Iteraciones	Loss final
Sigmoide (logistic)	0.8657	0.8400	485	0.3071
ReLU	0.8600	0.8533	406	0.2870

Análisis

El comportamiento inferido desde la representación gráfica (Figura 1.4) se corrobora en las métricas resultantes (Tabla 1.2): dada la comparativa en la iteración sobre este conjunto, al finalizar su rutina de ajuste el modelo inicializado con ReLU emplea un conteo de **406 iteraciones**, aventajando el cierre registrado por su homólogo activado bajo función Sigmoide programado a **485** ejecuciones cíclicas en sus épocas conjuntas.

Esta diferencia operativa se explica frecuentemente citando los problemas subyacentes derivados del atenuamiento por *vanishing gradient* (desvanecimiento del gradiente). Valores fuertemente alejados de su eje derivado obligan a la topología Sigmoidal a tender repetitivamente al valor de contigüidad base próximo a nivel estadístico asintótico cero; esto degrada las actualizaciones aplicadas operando en *backpropagation* con demoras remarcables para sus capas iniciales. Operando en su dominio positivo por otro lado, un hiperplano modelado a partir de nodos ReLU devuelve como valor de respuesta directa su propia constante limitando derivativamente retenciones para los pasos posteriores del escalado algorítmico, y favoreciendo ciclos uniformes sin congelación de coeficientes.

Al limitarse las iteraciones propuestas dentro de simulaciones genéricas y relativamente fáciles, el contraste frente a generalizaciones (pruebas aplicadas) ofrece una desviación moderada a la hora de arrojar aproximaciones válidas en los rangos cercanos a ~84-85 % para dos dimensiones. Aún así bajo arquitecturas formales complejas en escalas múltiples el desequilibrio matemático originado en bases sigmoidales produce caídas críticas justificadas para que se de total normalidad y aceptación al uso genérico y normalizado de unidades ReLU.

1.4. Tarea 4: Reconocimiento de Dígitos y Simplicidad de la Arquitectura

Como prueba final de diseño de un MLP, el objetivo planteado fue superar una precisión impuesta en validación por encima del 95 % (o superior) sobre la base de datos de imagen matricial `load_digits`, manteniendo siempre las dimensiones volu-

métricas de la topología en sus promedios factibles estables. Para ello, se iteraron de forma escalada un grupo de seis conjuntos diferentes sobre las activaciones probadas usando el optimizador estandarizado Adam.

Tabla 1.3: Métricas obtenidas para distintas arquitecturas en el dataset de dígitos.

Arquitectura	**Acc. Train**	**Acc. Test**	**Iteraciones**	**Profundidad**	**Meta**
MLP(20)	1.0000	0.9644	276	1 capa	✓
MLP(50)	1.0000	0.9711	180	1 capa	✓
MLP(100)	1.0000	0.9800	147	1 capa	✓
MLP(30, 15)	1.0000	0.9600	161	2 capas	✓
MLP(50, 20)	1.0000	0.9756	131	2 capas	✓
MLP(64, 32)	1.0000	0.9667	110	2 capas	✓

1.4.1. Análisis de sobreajuste

Al revisar los resultados en la Tabla 1.3, se destaca una tendencia generalizada al ajuste total frente al subconjunto de entrenamiento, donde **todas las variantes registran de partida un índice asimilado de un cien por ciento** (`acc_train = 1.0000`) dentro del volumen del dominio de validación. Esta asimetría manifiesta una fuerte tendencia experimental al sobreaprendizaje o curva de memorizado; no obstante es destacable de igual valor que tras este pico anómalo, la respuesta expuesta para evaluación generalista mantiene varianzas marginales promediando rangos paralelos a sus márgenes inferiores (rango variable desde 2 a 5 puntos absolutos sobre el muestreo test).

Dicho de otro modo, esta curva inicial en redes de excesiva resolución bajo series controladas de unos escasos miles de matrices (1.350 *samples*), es previsible y tratada a un margen favorable, tras dejar aplicado en este test un parámetro de amortiguamiento interno dentro de modulo de origen (‘sklearn‘ configurado en `alpha=1e-4`) obligando mediante norma L2 a impedir que este modelo rebasase los índices del factor paramétrico perdiendo por efecto correlativo viabilidad real para su correcta validación general en la red artificial.

1.4.2. Elección del modelo

Aun cuando arquitecturas densas en series ampliadas probadas de gran capa ($MLP(100)$) recogen en el último ciclo variaciones asimétricas arrojando hasta el 98 %, en un criterio general técnico orientado al desarrollo de sistemas eficientes la elección para este clasificador designaría como idónea a las capacidades del bloque

de características **MLP(20)**. Este marco de partida representa a grandes rasgos un sistema primario en orden para consolidar por entero el bloque de requerimientos fijados aportando más del 95 % empleando no un volumen sino parte de una fracción general contrapuesta de base calculada en ~1.500 parámetros teóricos reales si esto se contrapone con sus equivalentes de test ampliado bajo cuotas equivalentes que escalan y demandan de sus topologías sobre los teóricos aproximados 7.500 procesos elementales.

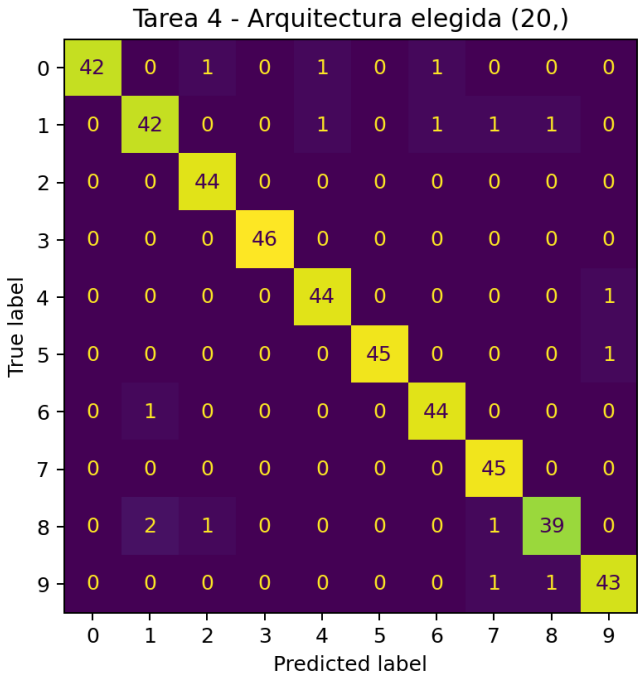


Figura 1.5: Matriz de confusión de la arquitectura **MLP(20)**. El desempeño es casi impecable en todas las clases; los escasos errores se concentran en pares de dígitos visualmente similares.

Propuesta Técnica de Despliegue

A la atención de: Equipo de Dirección
De: Área de Ingeniería y Data Science
Asunto: Recomendación de arquitectura para integración productiva

Finalizada la validación experimental, se procede a formular la propuesta técnica de despliegue para el nuevo sistema clasificador destinado a automatizar y agilizar el procesamiento interno:

1. Modelo Seleccionado

Tras contrastar diferentes arquitecturas, se emite recomendación para desplegar de forma base el modelo **MLP(20)** conformado por una capa discreta regular. Alcanza de modo formal su techo nominal probando el **96.44 %** de fiabilidad referida validada sobre conjuntos nuevos testados de serie analizada final, cumpliendo con certidumbre el requerimiento mínimo planteado para su consideración teórica o pase práctico general (95 %).

2. Justificación Técnica Estructural

Las validaciones demuestran la superior fiabilidad en marcos limitados sin la incursión no medida al implementar esquemas extendidos sin ganancial de impacto analítico. Pese a que una aproximación orientada de red $MLP(100)$ genera repeticiones finales estimadas en ~98 %, esta consume una aproximación contrapuesta con factor multiplicado exigiendo al hardware operando con volúmenes aumentados paramétricos de cinco ejes estructurales. Contar en base base con capas más restringidas pero igualmente eficientes repercute con margen para este tipo de topologías resultando en:

- **Mejores tasas de respuesta general (Latencia):** Entregas más estables durante el proceso continuo para llamadas cruzadas contra servidores API.
- **Consumos minoritarios operando memoria:** Asegurando con ello compatibilidades a un alto nivel de exigencia frente a integradores finales u otros desarrollos nativos y empotrados operados en ramas (Edge Computing).
- **Gestión y respuesta a la deriva del tiempo (Data Drift):** Los volúmenes sobre entrenados bajo una sobrepoblación computacional experimentan inestabilidades graves perdiendo todo margen analítico contra oscilaciones visuales derivadas bajo las integraciones de producción externa y de calle con mayor volumen de alteraciones estáticas a asimilar de la serie procesada en tiempo de muestreo real.

3. Consideraciones de Monitorización

A fin de delimitar su evaluación estática general resulta imperativo reiterar el índice sobreaprendido del clasificador en un cien por ciento base sobre los muestreos directos iniciales. A pesar que, los modelos formales en bloque compensan con efectividad la generalización validada tras el test recurriendo de base al modelo paramétrico limitador aportado por su propia penalidad interna de norma L2 en coeficientes asimilados, la viabilidad estructural debe completarse por una inserción

formal evaluada bajo una modalidad tipo **shadow deployment** (modo en sombra) estandarizado. Durante sus periodos tempranos esto validara en curso comparado el flujo directo natural a ser resuelto validando su generalización estática frente al tráfico real no supervisado del mundo de producción en vivo sin riesgo operacional a gran escala.